C – Comparison & Logical Operators

Dieses Dokument fasst alle Vergleichs- und logischen Operatoren in C übersichtlich zusammen. Der Fokus liegt auf Verständnis, Syntax und typischen Beispielen für Prüfungen.

1. Comparison Operators (Vergleichsoperatoren)

Operator	**Bedeutung**	**Beispiel**
==	gleich	a == b
!=	ungleich	a != b
<	kleiner als	a < b
>	größer als	a > b
<=	kleiner oder gleich	a <= b
>=	größer oder gleich	a >= b

⚠️ Wichtig: Der Vergleichsoperator == ist nicht dasselbe wie der Zuweisungsoperator =.

2. Logical Operators (Logische Operatoren)

Operator	**Bedeutung**	**Beispiel**
&&	UND (AND)	a > 0 && b > 0
||	ODER (OR)	a < 0 || b < 0
!	NICHT (NOT)	!fertig

C – if, switch & while (PLF-Zusammenfassung)

Dieses Dokument enthält alles, was du für die PLF zu if, switch und while in C brauchst. Es ist kompakt, übersichtlich und prüfungsorientiert aufgebaut.

1. if – else if – else

Grundstruktur:

```
if (Bedingung) {
   Anweisungen;
} else if (Bedingung) {
   Anweisungen;
} else {
   Anweisungen;
}
```

Beispiel:

```
if (note == 1) {
   printf("Sehr gut");
} else if (note == 2) {
   printf("Gut");
} else {
   printf("Nicht bestanden");
}
```

Nested if (verschachtelt):

```
if (zahl > 0) {
   if (zahl % 2 == 0) {
      printf("positiv und gerade");
   }
}
```

Typische Fehler:

- ❌ if (x = 5)
- ✅ if (x == 5)

2. switch – case

Grundstruktur:

```
switch (variable) {
   case 1:
      Anweisungen;
```

```
    break;
  case 2:
    Anweisungen;
    break;
  default:
    Anweisungen;
}
```

Beispiel:

```
switch (tag) {
  case 1: printf("Montag"); break;
  case 2: printf("Dienstag"); break;
  default: printf("Ungültig");
}
```

Mehrere cases ohne break (z.B. Vokale):

```
case 'a':
case 'e':
case 'i':
case 'o':
case 'u':
  printf("Vokal");
  break;
```

Wichtig für Prüfungen:

- • switch nur mit int oder char
- • break verhindert Durchfallen
- • default fast immer sinnvoll

3. while-Schleife

Grundstruktur:

```
while (Bedingung) {
  Anweisungen;
}
```

Beispiel (Zähler):

```
int i = 1;
while (i <= 5) {
  printf("%d ", i);
  i++;
}
```

Beispiel (Eingabevalidierung):

```
int x;
do {
   scanf("%d", &x);
} while (x < 1 || x > 10);
```

Analyse-Fragen (PLF-typisch):

- • Wie oft läuft die Schleife?
- • Wann endet sie?
- • Welche Werte nimmt die Variable an?

Typische Fehler:

- ❌ while (x > 0 || x < 10)
- ✅ while (x > 0 && x < 10)

4. Merksätze für die PLF

• if wird verwendet, wenn Bedingungen geprüft werden.
• switch ist ideal für viele feste Werte.
• while läuft, solange die Bedingung wahr ist.
• Jede Schleife braucht eine Änderung der Bedingung!
• Endlosschleifen sind fast immer Fehler.

Using `#define` for Constants

In C, you can define **constants** using the **preprocessor directive** `#define`.

The **`#define`** directive tells the **C preprocessor** to replace every occurrence of a defined name with its value **before the program is compiled**. This happens **textually**, meaning the compiler never actually sees the names — only the substituted numbers or text.

This means that before the compiler starts working, the C preprocessor replaces the names of the constants by their values.

Example:

Before the preprocessor runs

```
#define NUMBER 10

int main() {
    int a = NUMBER;

    return 0;
}
```

After the preprocessor ran

```
int main() {
    int a = 10;

    return 0;
}
```

This is useful for values that should not change during program execution, such as limits or bounds for random numbers.

To make `gcc` only run the preprocessor, you may call `gcc -E filename.c`.

The syntax is:

```
#define NAME value
```

For example:

```
#define UPPER_BOUND 10
#define LOWER_BOUND 5
```

Let us consider the following program:

```
#include <stdio.h>

#define METER_TO_FOOT 3.28084
#define NUM_LETTERS ('a' - 'A')

int main() {
    double meters = 0.0;
    double feet = 0.0;
    char small = '0';
    char capital = '0';

    // Read a length in meters
    printf("Enter length in meters: ");
    scanf("%lf", &meters);
```

```
    // Convert meters to feet
    feet = meters * METER_TO_FOOT;
    printf("%.2f meters = %.2f feet\n", meters, feet);

    // Read a lowercase letter
    printf("Enter a small letter (a-z): ");
    scanf(" %c", &small);   // space before %c to skip any whitespace

    // Convert to uppercase
    capital = small - NUM_LETTERS;
    printf("The capital letter is: %c\n", capital);

    // Display constant (optional, for illustration)
    printf("There are %d letters in the alphabet.\n", NUM_LETTERS);

    return 0;
}
```

Output

```
Enter length in meters: 2.5
2.50 meters = 8.20 feet
Enter a small letter (a-z): g
The capital letter is: G
There are 26 letters in the alphabet.
```

It looks like this after the preprocessor ran and before the code gets compiled:

```
// #include <stdio.h> is replaced by A LOT of code!!!

int main() {
    double meters = 0.0;
    double feet = 0.0;
    char small = '0';
    char capital = '0';

    // Read a length in meters
    printf("Enter length in meters: ");
    scanf("%lf", &meters);

    // Convert meters to feet
    feet = meters * 3.28084;
    printf("%.2f meters = %.2f feet\n", meters, feet);

    // Read a lowercase letter
    printf("Enter a small letter (a-z): ");
    scanf(" %c", &small);   // space before %c to skip any whitespace

    // Convert to uppercase
    capital = small - ('a' - 'A');
    printf("The capital letter is: %c\n", capital);

    // Display constant (optional, for illustration)
    printf("There are %d letters in the alphabet.\n", ('a' - 'A'));

    return 0;
}
```

Random Number Generation in C

Introduction

Computers cannot produce truly random numbers — they generate **pseudo-random numbers** that appear random but are calculated by a deterministic algorithm.
The C standard library provides functions for this in **`stdlib.h`** (*and* `time.h`).

To start, include the necessary header file:

```
#include <stdlib.h>
#include <time.h>
```

Initializing the Random Number Generator

The `rand()` function generates pseudo-random integers. To make the sequence different each time the program runs, use:

```
srand(time(NULL));
```

`time(NULL)` returns the current system time (in seconds), which serves as a changing seed value for the generator.

RAND_MAX

`RAND_MAX` is a constant defined in `stdlib.h` (like `M_PI` in `math.h`) that specifies the largest value the function `rand()` can return. Every call to `rand()` produces an integer from `0` to `RAND_MAX` (inclusive).

The exact value of RAND_MAX depends on the compiler and system — it's often `32767` on many implementations, but can be much larger (for example, `2147483647` on modern systems).

Generating Integer Random Numbers

1. <u>From `0` to `RAND_MAX`</u>

   ```
   int r = rand();
   ```

   This produces a number in the range **0 … RAND_MAX**, where RAND_MAX is a constant defined in <stdlib.h> (usually 32767 or higher).

2. <u>From `0` to a given upper bound</u>

   ```
   #define UPPER 10
   …
   int r = rand() % (UPPER + 1);
   ```

This gives numbers from **`0, 1, 2, …, UPPER`**. All values are evenly distributed modulo the upper bound + 1.

3. In a custom interval `[LOWER_BOUND, UPPER_BOUND]`

```
#define LOWER -5
#define UPPER 22
…
int r = LOWER + rand() % (UPPER - LOWER + 1);
```

The expression shifts the range from starting at **`0`** to starting at **`LOWER`**, ensuring the values stay between **`LOWER`** and **`UPPER`**, inclusive.

Generating Floating-Point Random Numbers

1. From `0.0` to `1.0`

```
double rd = rand() / (double)RAND_MAX;
float rf = rand() / (float)RAND_MAX;
```

Dividing by `RAND_MAX` converts the integer to a floating-point value between `0.0` and `1.0`.

2. In a floating-point interval

```
double rf = LOWER + (rand() / (double)RAND_MAX) * (UPPER - LOWER);
```

Produces a random floating-point number between `LOWER` and `UPPER`.

Example program

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define LOWER_INT -5
#define UPPER_INT 12

#define LOWER_FLOAT 7.2
#define UPPER_FLOAT 10.7

#define LOWER_CHAR 'C'
#define UPPER_CHAR 'R'

int main()
{
    int randomInt = 0;
    float randomFloat = 0.f;
    char randomChar = '0';

    // Initialize random number generator
    srand(time(NULL));

    randomInt = LOWER_INT + rand() % (UPPER_INT - LOWER_INT + 1);
```

```
    printf("Integer random number in [%d;%d]: %d\n",
       LOWER_INT, UPPER_INT, randomInt);

    randomFloat = LOWER_FLOAT + (rand() / (float)RAND_MAX) *
       (UPPER_FLOAT - LOWER_FLOAT);
    printf("Float random number in [%.2f;%.2f]: %.2f\n",
       LOWER_FLOAT, UPPER_FLOAT, randomFloat);

    randomChar = LOWER_CHAR + rand() % (UPPER_CHAR - LOWER_CHAR + 1);
    printf("Random letter in [%c;%c]: %c\n",
       LOWER_CHAR, UPPER_CHAR, randomChar);

    return 0;
}
```

Output

```
Integer random number in [-5;12]: 12
Float random number in [7.20;10.70]: 10.03
Random letter in [C;R]: K
```

Erzeugen von ganzzahligen Zufallszahlen:

Um Zufallszahlen zu erzeugen, benötigt man folgende Funktionen:

`srand(time(NULL)):` definiert den Startwert zum Erzeugen der Zufallszahl abhängig von der aktuellen Zeit

`rand():` liefert eine ganzzahlige Zufallszahl aus dem Bereich 0 bis RAND_MAX (meist 32767); Über den Modulo-Operator kann man diesen Bereich eingrenzen.

```
#include <time.h>

#include <stdlib.h>

int main(){

  int randomNumber = 0;

  int lowerBound = 25;

  int upperBound = 50;

  int range = 0;


  // Initialisierung des Zufallszahlengenerators abhängig von der aktuellen Zeit

  srand(time(NULL));


  // Erzeuge eine Zufallszahl aus dem Intervall [0; MAX_INT].

  randomNumber = rand();


  // Erzeuge eine Zufallszahl aus dem Intervall [0; 9].
  // D.h. es sind 10 verschiedene Zufallswerte möglich.  → (0, 1, … , 9)

  randomNumber = rand()%10;


  // Erzeuge eine Zufallszahl aus dem Intervall [1; 10].
  // D.h. es sind 10 verschiedene Zufallswerte möglich.
  // Da der Bereich durch die %-Rechnung bei 0 beginnt, muss man
  // zum Zufallswert noch den Wert 1 addieren.
  // (0, 1, … , 9)  → (1, 2, … , 10)

  randomNumber = rand()%10 + 1;


  // erzeuge eine Zufallszahl aus dem Intervall [10; 20]
  // D.h. es sind 11 verschiedene Zufallswerte möglich.   →  20 - 10 + 1 = 11
  // Da der Bereich durch die %-Rechnung bei 0 beginnt, muss man
  // zum Zufallswert noch den Wert 10 addieren.
  // (0, 1, … , 10)  → (10, 11, … , 20)

  randomNumber = rand()%(20-10+1) + 10;
```

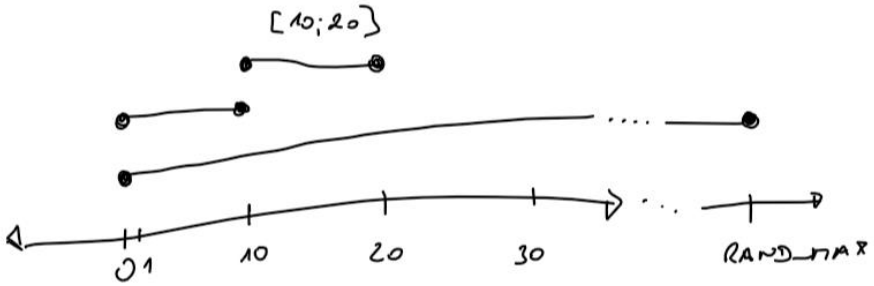
```
  // Erzeuge eine Zufallszahl im Bereich von [-15; +15].
  // D.h. es sind 31 verschiedene Zufalsswerte möglich.   →  15 - (-15) + 1 = 31
  // Da der Bereich durch die %-Rechnung bei 0 beginnt, muss man
  // zum Zufallswert noch den Wert -15 addieren.
  // (0, 1, … , 30)  → (-15, -14, … , 15)

  randomNumber = rand()%(31) - 15;


  // Erzeuge eine Zufallszahl im Bereich von [lowerBound; upperBound].

  randomNumber = rand() % (upperBound - lowerBound + 1) + lowerBound;

  // oder aufgeteilt:

  // range = upperBound - lowerBound + 1;
  // randomNumber = rand() % range + lowerBound;


  return 0;

}
```

$zz \in [10; 20]$

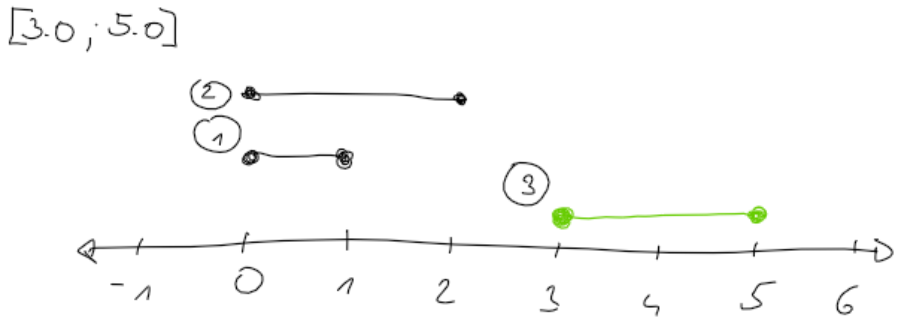
$\underbrace{rand()}_{0 \le zz \le RAND_MAX} \quad \% \quad 11 \quad + \quad 10;$

$20 - 10 + 1 = 11$

$\underbrace{\underbrace{0 \le zz \le 10}_{10 \le zz \le 20}}$



Gleitkommazufallszahlen




```
double randomNumber = 0.0;
double begin = 3.0;
double end = 5.0;
double range = 0.0;

srand(time(NULL));
range = end - begin;
```

siehe Anmerkung

$$\text{randomNumber} = \underbrace{\text{begin} + \underbrace{\underbrace{\left(\underbrace{(\text{double})\ \text{rand}()}_{[0;\ \text{RAND_MAX}]}\ /\ \text{RAND_MAX}\right)}_{[0{,}0;\ 1{,}0]\ \ (1)}\ *\ \text{range}}_{[0{,}0;\ 2{,}0]\ \ (2)}}_{[3{,}0;\ 5{,}0]};$$

Anmerkung:

Um eine Gleitkommadivision zu erreichen, gibt es folgende Möglichkeiten:

1. (double) rand() / RAND_MAX
2. rand() / (double) RAND_MAX
3. rand() / (RAND_MAX * 1.0)
4. 1.0 * rand() / RAND_MAX

Variables in C

Introduction

In C, every variable must have a **data type** that tells the compiler what kind of data it stores. A variable is a **named storage location in memory**.

Declaring Variables

Before using a variable, you must **declare** it with a data type and a name:

```
int age;
float temperature;
double weight;
char letter;
long population;
```

You can also **initialize** variables directly with a value:

```
int age = 20;
float temperature = 36.5f;
float weight = 80.512;
char letter = 'A';
long population = 123456789L;   // suffix L for long literal
```

A general rule: Always initialize variables!

Basic Data Types in C

Data type	Description	Example values	Typical range*
char	Single character (ASCII code)	'A', '7'	0 … 255
int	Integer (whole numbers)	-3, 0, 42	-2,147,483,648 … 2,147,483,647 (32-bit)
long	Larger integer)	100000, -2500000	9,223,372,036,854,775,808 … 9,223,372,036,854,775,807 (64-bit)
float	Floating-point number (≈ 7 digits precision)	3.14, -0.5	~ ±3.4 × 10^38
double	Double-precision floating-point (≈ 15 digits)	3.141592	~ ±1.7 × 10^308

*Ranges depend on compiler and platform, but these are typical values.

Literals

A **literal** is a fixed value written directly in the code.

Examples:

- **Integer literals**: `10`, `-25`, `0`
  - Add suffix **`L`** for `long` (`1000000`**`L`**)
- **Floating-point literals for `float`**: `3.14`**`f`**, `-0.001`, `2.0e3`**`f`** (scientific notation)
- **Floating-point literals for double**: `3.14`, `-0.001`, `2.0e3` (scientific notation)
- **Character literals**: `'A'`, `'7'`, `'\n'` (escape sequence for newline)

Example Program

```
int number = 42;                // integer literal
long big = 1000000L;            // long literal
float pi = 3.14f;               // float literal (f suffix)
double e = 2.7182818284;        // double literal
char newline = '\n';            // character literal
```

```
#include <stdio.h>

int main() {
    int number = 10;
    long population = 8000000L;
    float pi = 3.14f;
    double precise = 3.1415926535;
    char letter = 'C';

    printf("Number: %d\n", number);
    printf("Population: %ld\n", population);
    printf("Pi: %f\n", pi);
    printf("Precise Pi: %.10f\n", precise);
    printf("Letter: %c\n", letter);

    return 0;
}
```

Output:

```
Number: 10
Population: 8000000
Pi: 3.140000
Precise Pi: 3.1415926535
Letter: C
```

Variables in Memory

Each variable occupies space in memory. With the **address-of operator `&`**, you can print the memory address of a variable:

```
printf("%p", &population); // prints the memory address of 'population'
```

Worksheet – log. Expressions, if/else – 1DHIF 2025/26 NAME: _____

Exercise 1: Let `A`, `B`, `C` be variables which are either `true (1)` or `false (0)`. Complete the table, such that you can determine the truth values of **`A && B && (!A || !(B || C))`**.

A	**B**	**C**	**A && B**	**!A**	**B || C**			**A && B && (!A || !(B || C))**
0	0	0						
0	0	1						
0	1	0						
0	1	1						
1	0	0						
1	0	1						
1	1	0						
1	1	1						

Exercise 2: Analyze the code and write the output next to the `printf`s. Reason, which part of an expression with multiple operators (e.g. **`a!=b && c>d;`**) gets evaluated („ausgewertet“) first and write down the intermediate results (Zwischenergebnisse)

```
#include<stdio.h>

int main() {
    int a = 10; int b = 20; char c = 'A'; long d = -123L; int result = 0;

    result = a > b;
    printf("%d\n", result);     // ________________________

    result = -111L || 0;
    printf("%d\n", result);     // ________________________

    result = c && 0;
    printf("%d\n", result);     // ________________________

    result = a!=b && c>d;
    printf("%d\n", result);     // ________________________

    result = (a==b || c<255) && d>b;
    printf("%d\n", result);     // ________________________

    result = -10 * ((-10>0) - (-10<0));
    printf("%d\n", result);     // ________________________

    if(-10) {
        if(!0) {
            printf("foo\n");    // Steht „foo“ am Bildschirm? Ja/Nein?
        }
    }

    return 0;
}
```

Exercise 3: Those are the stats of your avatar in a role-playing game:

`int age = 17; float height = 1.75f; int isHealthy = 0;`

`float cash = 45.f; int deceiveSkill = 70;` (to deceive = täuschen)

Your avatar is allowed to enter the tavern (next quest!) if and only if the conditions below hold. Evaluate (werte aus) the expression step-by-step and write down the intermediate results (`0` or `1`) below the `( )`!

```
(age >= 18 && height > 1.50f && !isHealthy) || (cash >= 20.f && deceiveSkill > 50)
```

Exercise 4: In a program's source code, there can be various types of errors, including so-called **logical errors** and **syntactic errors**. (The syntax of a programming language describes which combinations of characters form valid programs in that language. It is essentially the grammar of the programming language.)

Consider the following two sentences from the German language:

- I very much to like the subject POS.
- When a student writes a perfect PLF, then it is graded as "insufficient".

One of the sentences contains a **logical error**, and the other contains a **syntactic error**.

Which sentence contains which error?

The following program contains both logical and syntactic errors. **Fix them!**

```
#include<stdio.h>

int main() {
    double temperature = 23.2; // Should be at least 20°.
    double co2 = 0.1;          // Should be less than 0.1.

    if(temperature >= 20.0 & co2 < 0.1) {
        printf("Everything okay.\n");
    } else if(temperature <= 20.0 && co2 < 0.1); {
        printf("Please turn up the heater.\n");
    } else if(temperature < 20.0 && co2 >=0.1)
        printf("Open the window and turn up the heater aftewards.\n");
    } else (temperature > 20.0 && co2 < 0.1) {
        printf("Please open the window.\n");
    }

    return 0;
}
```

Exercise 5: Implement the following rules correctly using `if/else` or nested `if` statements:

The program should check whether `number` is even or odd only if `number` is greater than `0`. Depending on the result, "`even number`" or "`odd number`" should be displayed on the screen.

If `number` is less than `0`, the square of the `number` should be displayed on the screen.

```
    // …
    int number = 12;









   // …
```

Andreas Kucher

Zusammenfassung: Ein- und Ausgabe am Bildschirm

Inhalte: Text mit `printf` ausgeben, Escape-Sequenzen, Ausgabe von Variablen, formatierte Ausgabe, Einlesen von Zahlen

Vorsicht:

`&` bei `scanf`, `fflush(stdin)` oder `%c` beim Einlesen von Zeichen, Ausgabe von \ -> \\, „ -> \", ...

Ausgabe über den Bildschirm und Escape-Sequenzen

- `printf("Text\n");` -> Text in Hochkommata, `\n` -> neue Zeile
- weitere Escape-Sequenzen: `\t` (Tabulator), `\\` -> Backlash, `\?` Fragezeichen, ...
- Beispiel
```
printf("Hello World!\n\tDas ist ein Text");
     Hello World!
          Das ist ein Text
```

Ausgabe von Variablen

`%[flag][width][.precision]conversion` (Angaben in `[]` sind optional)

- `Conversion`
  - `%c` -> ein Zeichen
  - `%d` -> `int`
  - `%ld` -> `long`
  - `%f` -> `float`
  - `%lf` -> `double`
  - `%c` -> `char`
- `flag`
  - – -> linksbündig
  - + -> immer mit Vorzeichen
  - `0` -> führende Nullen
- `width`
  - Minimale Breite in Zeichen
- `.precision`
  - Für Gleitkommazahlen; wie viele Nachkommastellen werden gezeigt?
  - Standardmäßig 6 Nachkommastellen
- Beispiel
```
float f = 342.23322f; double d = 123.444455;
printf(„%+5.2f %-010.3d, f, d); // Ausgabe
```

Einlesen von Variablen

- Befehl: `scanf`
- `conversion` wie bei `printf`
- **Bei den Variablen muss ein `&` stehen!**
- Beim Einlesen eines Zeichens: vorher `fflush(stdin);` **oder Leerzeichen vor `%`**
- Beispiel
  - `char c; double d; scanf(" %c %lf", &c, &d);`